Chapter 4

DECOMPOSITION MODELS

Decomposition models describe solids through a combination of some basic building blocks glued together. The kinds on basic objects used and the way the combination of basic objects is recorded leads to variations on this basic theme.

4.1 EXHAUSTIVE ENUMERATION

Earlier, we viewed solids as continuous point sets. While we cannot list all points belonging to a solid, we can easily list all, say, tiny cubes that are contained (completely or partially) in the solid. The cubes are assumed to be nonoverlapping and of uniform size and orientation, i.e., they form a *regular subdivision* of the space.

The resulting solid representation is called the *exhaustive enumeration*; see Figure 4.1 [28]. In the collection of cubes forming the exhaustive enumeration, each small cube can be completely described in terms of its corners. By regularity, it is sufficient to store the coordinates of just one corner for each cube of the collection. In the case that we can *a priori* limit our interest to some *space of interest* being a subset of E^3, the collection can be efficiently encoded as a three-dimensional array c_{ijk} of binary data. The array represents the "coloring" of each cube: if $c_{ijk} = 1$ ("*black*"), the cube ijk represents a solid region of the space, and otherwise it is empty ("*white*").

The binary array representation is, of course, the obvious way to represent a picture in *digital image processing*. Reference [112] surveys object

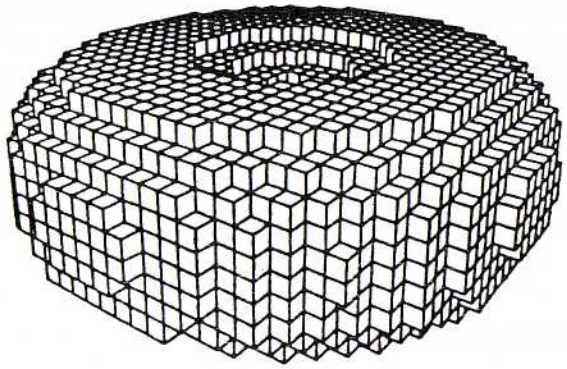

Figure 4.1 Exhaustive enumeration.

representations of this field; see also [137]. While image processing as such is beyond the scope of this book, the areas of interest of image processing and solid modeling do intersect, and much of what we shall say on the processing of exhaustive enumerations (or decomposition models in general) has its roots in image processing techniques.

4.1.1 Construction of Exhaustive Enumerations

An independent solid description mechanism for exhaustive enumerations would consist of a list of all cubes that are considered to form parts of the solid. In image processing applications this is indeed possible with image scanning devices. Digital tomography [123] can also supply three-dimensional digital information.

Unfortunately, in solid modeling we rarely have an image or an object to start with, and these direct solid description techniques cannot be used. It would also be very hard to describe realistic objects as directly enumerating its cubes. A more practical approach is to create exhaustive enumerations by a conversion from some other representation such as the constructive or boundary models to be discussed in the subsequent chapters. Other objects can then be created by Boolean set operations or other suitable algorithms for exhaustive enumerations.

The interesting point of exhaustive enumeration is that it is straightforward to produce algorithms that create new representations from existing representations. A prime example is the calculation of Boolean set operations on exhaustive enumerations. In the binary matrix representation, the algorithm becomes just the corresponding bitwise operation for all corresponding elements.

Simplistically, a binary matrix is always a valid solid representation; hence, all algorithms that produce them can be considered closed. However, often disconnected cells that do not have any neighbor cells are not desired, and algorithms such as the Boolean set operations must pay attention to disallowing them. Quite similarly, a single white cell within a block of black cells might not be considered correct. For various situations, various degrees of connectivity may be desired. Each 3-dimensional cell has six face neigbors, 12 edge neighbors, and eight vertex neighbors. The strictest connectivity criterion would require that all black cells have at least one black face neighbor.

Finding and filling connected components (with various degrees of connectivity) are standard image processing algorithms [92]. It might be possible to apply image processing techniques for solid modeling purposes also more generally. For instance, a *growing algorithm* (*dilatation*) could be used to calculate the "offset solid" of a given solid, i.e., the solid that contains all points at distance $d < r$ from any point of the original solid. This operation would be useful for the generation of data for numerically controlled (NC) machine tools and robotics.

4.1.2 Uses of Exhaustive Enumeration

Two- and three-dimensional variants of the exhaustive enumeration are the obvious representations for digital images, and are widely used in digital image processing. Through this connection, exhaustive enumeration has found its solid modeling uses in special applications, such as modeling of cell particles. In this area, a cube is allowed to have other "colors" beside mere "white" or "black," again raising the memory consumption.

Exhaustive enumeration is as much a representation of empty spaces as of spaces occupied by material. A coloring scheme can be used to distinguish various kinds of empty spaces; this is useful, for instance, for modeling buildings in heat transfer analysis. In this context, empty spaces in the interior and in the exterior of the building must be modeled differently.

Exhaustive enumeration is also used as an auxiliary scheme for speeding up operations on other representations. For instance, while being based on the half-space approach to be described in the next chapter, the solid modeler TIPS [91] uses a regular three-dimensional grid as a geometric directory to the elements of the half-space model in order to speed up various geometric algorithms. We shall return to this in Chapter 18.

4.1.3 Properties of Exhaustive Enumeration

In summary, let us discuss the properties of the exhaustive enumerations according to the framework of Section 3.6.1.

Expressive power: Exhaustive enumeration is obviously *approximative*: surfaces that are not coplanar with any of the coordinate planes of the subdivision will be only approximately represented. With this restriction, however, the scheme is general: all kinds of objects can be represented under it.

Validity: The validity of exhaustive enumerations depends on the geometric interaction of individual cubes. Specifically, each pair of blocks may intersect only at a common vertex, edge, or face. If the binary matrix representation can be used, all exhaustive enumerations are valid if connectivity is not required.

Unambiguity and uniqueness: All valid exhaustive enumerations are unambiguous. They are also *unique*: in a fixed space of interest and resolution, each solid has just one representation.

Description languages: In image processing applications, exhaustive enumerations are generally created through scanning of an image. In the context of solid modeling proper, their generation is ordinarily based on conversion from other models.

Conciseness: Exhaustive enumerations tend to be fairly large: a resolution of 256^3 (which is only barely adequate) takes 16 million bits of storage. Storage requirements rise sharply with increased resolution.

Closure of operations: Exhaustive enumerations support many closed algorithms. Boolean set operations are a prime example.

Computational ease and applicability: Algorithms for this scheme tend to be extremely simple; however, the mere size of the object representations means that they are slow in an ordinary computer. Note, however, that the computation required is typically extremely *regular*: the calculation needs to consider just one cube or at most the cube and a few neighbor cubes. This regularity means that these representations are prime candidates for VLSI implementation. This can radically affect the applicability of exhaustive enumeration in the future.

4.2 SPACE SUBDIVISION SCHEMES

Exhaustive enumerations have many virtues: they are simple, general, and allow the use of a wide variety of algorithms. These good points are, however, offset by the huge memory consumption and the mediocre accuracy possible. To overcome this, many representations replace the underlying

regular space subdivision of the pure enumeration by a more efficient, adaptive subdivision.

These *adaptive subdivision schemes* are based on the simple observation that in the three-dimensional grid of an exhaustive enumeration, the neighbor cubes of a white cube are very likely to be white as well. By encoding this information into one combined node of the data structure considerable savings over the complete grid are possible.

Adaptive subdivisions use the fundamental property that the number of nodes needed for the representation of a solid is proportional to its surface area [82]. Hence the number of elements is proportional to the square r^2 of the resolution r used, whereas the size of exhaustive schemes is proportional to r^3.

4.2.1 The Octree Representation

Prime examples of adaptive space subdivision schemes are the *Octree representation* for solid objects [60,82] and the analogous *Quadtree representation* [106] for two-dimensional objects.

The octree representation uses a recursive subdivision of the space of interest into eight *octants* that are arranged into an 8-ary tree (hence the name). Figure 4.2(a) depicts the octant subdivision.

Usually the octree is thought of as being located around the origin of its local xyz-coordinate system, with its first-level octants corresponding to the octants of that space, and in particular octant 3 being the positive octant $x, y, z > 0$ of the space. Hence it is necessary to represent the actual space of interest separately in terms of an appropriate transformation.

Each node of an octree consists of a *code* and eight pointers towards eight sons, numbered 0 through 7. If *code* = *black*, the part of the space represented by the node is all material and the pointers 0 through 7 are nil, i.e., the node is a leaf. If *code* = *white*, the part of the space is empty and the node is again a leaf. The third possibility *code* = *grey* corresponds to the case where the part of the space is partly material and partly empty. In this case, the eight pointers point to eight children that correspond to a regular subdivision of their parent node. For instance, the object of Figure 4.2(b) would be represented by the 2-level octree shown in (c).

Program 4.1 outlines a data structure for representing octrees. Here the space of interest is an orthogonal box of the xyz-space, represented by a special "root" node.

It is easy to show that 7/8 of the nodes of an octree are leaves. Because of this, also leaves are usually represented with a special node to avoid allocating storage for the eight nil pointers.

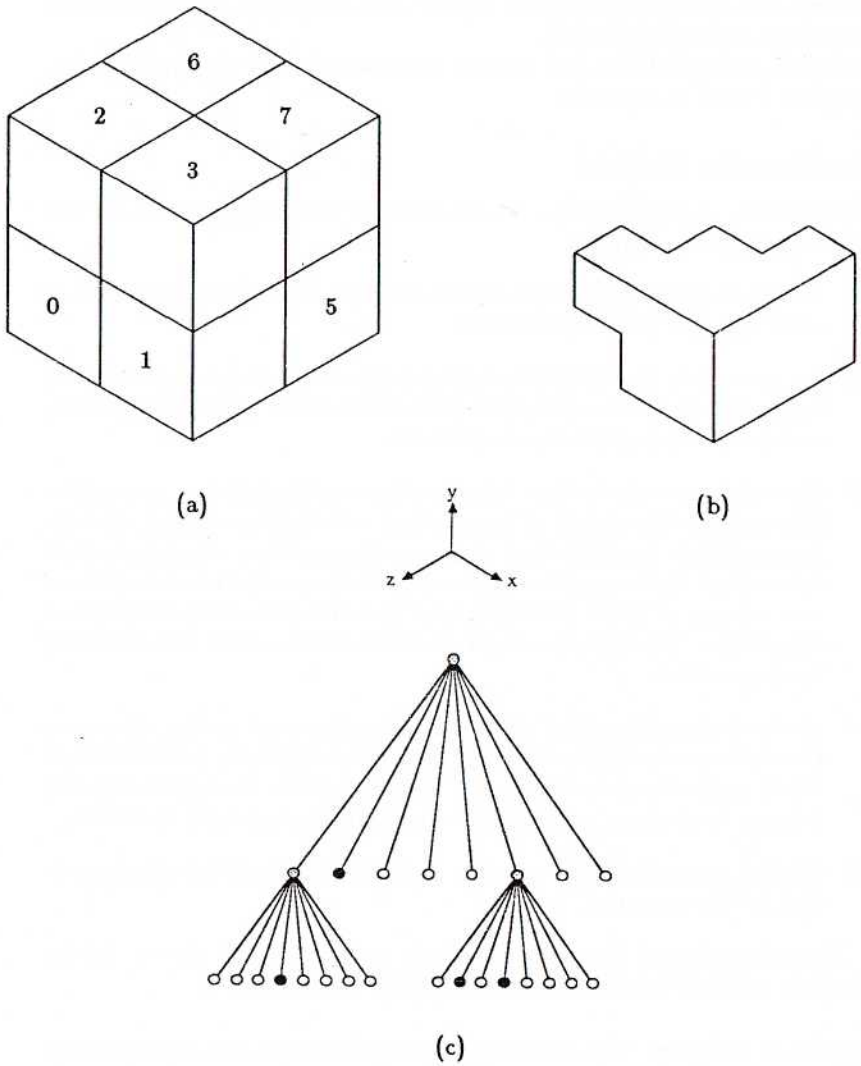

Figure 4.2 An octree model.


```
struct octreeroot
{
        float           xmin, ymin, zmin;  /* space of interest */
        float           xmax, ymax. zmax;
        struct octree   *root;             /* root of the tree */
};

struct octree
{
        char            code;              /* BLACK, WHITE, GREY */
        struct octree   *oct[8];           /* pointers to octants,
                                              present if GREY */
};
```

Program 4.1 Octree data structures.

Construction of Octree Representations

In the solid modeling context, octrees are ordinarily constructed from solid primitives. For each primitive type, a *classification procedure* between an instance of the primitive and an arbitrary node of the octree is needed. The classification must be capable of distinguishing between the following three cases:

1. The node considered is completely in the exterior of the primitive.
2. The node considered is completely in the interior of the primitive.
3. The node is partially in the interior of the primitive.

The classification procedure is used in a recursive fashion. Initially, the whole space of interest is represented by one node with *code* = *white*, and the algorithm is started at the single white node. Each node is classified against the primitive with the classification procedure. If the first or the second case above applies, the node is marked *white* or *black*, and the recursion terminates. Otherwise, the algorithm proceeds by marking the node *grey*, subdividing it into eight octants, and calling itself recursively for each octant. The subdivision is continued until a desired resolution has been reached, usually up to 6–12 levels.

The construction algorithm is outlined in Program 4.2. The actual solid primitives could be represented according to the primitive instancing scheme, i.e., in terms of a primitive type code and dimension and orientation parameters.

```
make_tree(p, t, depth)
primitive *p; /* p = the primitive to be modeled */
octree *t;    /* t = node of the octree, initially
                 the initial tree with one white node */
int depth;    /* initially max. depth of the recursion */
{
       int    i;
       switch(classify(p, t))
       {
           case WHITE:
               t->code = WHITE;
               break;
           case BLACK:
               t->code = BLACK;
               break;
           case GREY:
               if(depth == 0)
               {
                   t->code = BLACK;
               }
               else
               {
                   subdivide(t);
                   for(i=0; i<8; i++)
                        make_tree(p, t->oct[i], depth-1);
               }
               break;
        }
}

/* classify octree node against primitive */
classify(...)

/* divide octree node into eight octants */
subdivide(...)
```

Program 4.2 Construction of an octree.

The collection of possible primitives depends on whether a classification algorithm for the primitive can easily be implemented. Typical "easy" primitives include rectilinear blocks, half-spaces such as sphere, cylinder, and cone, and certain higher-order objects such as tori and objects bounded by so-called superquadrics [10].

Octrees and quadtrees can also be constructed from digital image information if such is available.

Algorithms for Octrees

A functionally complete octree modeler should include algorithms for the following categories of tasks:

1. *Tree generators* that create octrees from parameterized primitives or other types of geometric models.

2. *Set operations* that take two octrees (with identical spaces of interest) and calculate a new octree that gives the Boolean union, intersection, or set difference of the two arguments.

3. *Geometric operations* that take an octree and calculate a new octree that models the result of translating, rotating, or scaling the object modeled. Another type of geometric operation calculates a new octree that has been altered according to the *perspective transformation*. Some of these problems lead to unintuitive and complicated algorithms. For instance, the translation of an octree is a relatively hard operation.

4. *Analysis procedures* that calculate properties such as the volume or the surface area of an octree. A *connected components* procedure that labels each octree node with the identifier of the connected object it belongs to gives an example of a more involved analysis operation.

5. *Display generators* that create a graphical image of the object modeled by the octree.

The generation of octrees was already outlined in the above. In the following, we shall briefly discuss some of the other areas.

Graphical Output The essential property of octrees is that they record the shape information of an object in a spatially ordered manner. If this can be exploited in algorithm design, it is possible to create very simple algorithms that just scan their argument trees and perform relatively simple operations at each node.

Generation of graphical output for a frame buffer raster display is a particularly good example of this. Observe that by traversing the nodes of the octree in a suitable order, the parts of the image generated from far nodes can be painted before those of the near ones. Hence the close parts will overlay the far away parts in the frame buffer, and no sorting is required for the generation of an image with hidden surfaces removed.

The proper ordering depends on the location of the viewer with respect to the tree. For instance, if the viewer is located in the octant $x, y, z > 0$ of the space (such as in the view of Figure 4.2(a)), the ordering 4, 0, 5, 1, 6, 2, 7, 3 of the subtrees will produce the correct result. Reference [31] describes techniques for the generation of more advanced images based on this approach.

Analysis Many analysis operations can also be performed according to a similar node-by-node paradigm. For instance, integral properties such as volume, center of mass, and moments of inertia can all be calculated by simple tree traversal algorithms.

Set Operations for Octrees Also set operations for octrees lead to a tree traversal algorithm. The set operations algorithm takes two argument octrees, and produces a third octree that represents the desired Boolean set operation (union, intersection, set difference) of the arguments.

The argument trees are traversed in a synchronous fashion, and a case analysis as for the operation and the types of the corresponding nodes is performed. For instance, when calculating the set intersection, the following cases would occur in the processing of two corresponding nodes n_1 and n_2:

1. Nodes n_1 and n_2 are both leaves. In this case, the corresponding node of the result octree is black if both n_1 and n_2 are black; otherwise, it is white.

2. Either n_1 or n_2 is a leaf. In this case, if the leaf node is black, the subtree of the nonleaf is copied to the result octree. Otherwise, the node of the result tree is white.

3. Nodes n_1 and n_2 are both nonleaves. In this case, the algorithm considers recursively their children as above.

Observe that if the both octrees are already present (i.e., the time needed for their construction can be ignored), the complexity of this algorithm is at most proportional to the size of the smaller tree.

Limitations of tree traversal algorithms Unfortunately, not all algorithms for octrees can be brought in the form of a simple tree traversal. In graphics, visual effects such as smoothly shaded surfaces and transparency require the capability of accessing the neighbors of a node. Unfortunately, accessing a neighbor node can in the worst case require a traversal up to the root of the tree, and down to the neighbor. Similar lines apply also to various image processing algorithms such as "growing" and connected component labeling.

The complexity of accessing neighbor nodes can be somewhat cured by introducing further pointers in the octree data structure. As this raises the cost of the generation of octrees, careful analysis of the frequency of the various operations is needed for maintaining the proper balance.

Properties of Octrees

The properties of octree representations are similar to those of the exhaustive enumeration, with some noticeable exceptions.

Expressive power: As were exhaustive enumerations, octrees are approximative representations, and model exactly only rather peculiar objects. However, if a classification routine for a primitive can be written, arbitrarily accurate octrees for it can be generated (at the cost of high storage use).

Validity: If no special conenctivity requirement are posed, all octrees are valid representations of some solid.

Unambiguity and uniqueness: Up to the limits of resolution, all octrees unambiguously define a solid. On a fixed resolution, the representation is also unique: an object has just one compacted[1] octree representation with at most n levels.

Description languages: The same lines apply as for exhaustive enumerations. Octrees are usually formed by conversion from other representations, of which constructive representations have an especially important role. In image processing applications, quad- and octrees are also formed directly from rasterized image data.

Conciseness: In general, the number of nodes in the octree representation of a solid object is proportional to the surface area of the object [82]. Hence octree models are not quite as large as exhaustive representations but still take a fair amount of storage. An octree representation of an "average" engineering object easily takes more than 1 million bytes of memory.

[1] Algorithms such as set operations can create octrees with unnecessary nodes (e.g., an internal node whose children are all black). Such nodes can be removed with a relatively simple tree traversal algorithm.

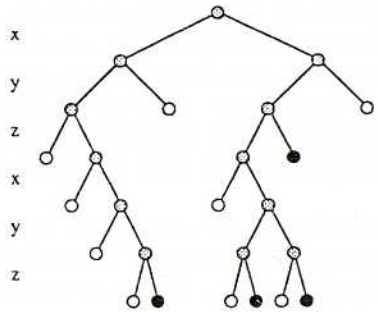

Figure 4.3 Binary subdivision tree.

Closure of Operations: Octree models support closed algorithms for problems such as translation, rotation, and Boolean set operations.

Computational ease and applicability: Many algorithms for octrees take the form of a tree traversal where a relatively simple operation is performed at each node of the tree. To these algorithms, the same notes apply as to exhaustive enumeration, including the potential role of VLSI.

Octree and quadtree representations have won considerable interest, and their literature is large and grows rapidly. For a comprehensive survey, the reader is referred to [106].

4.2.2 Binary Space Subdivision

As an alternative to the octree formulation, a *binary space subdivision* is also possible. As the name implies, this approach divides a grey node in two halves, instead of eight octants. The subdivision is performed successively in the direction of x, y, and z (see Figure 4.3).

In comparison with octrees, explicit binary subdivision trees are slightly smaller. As the virtues of the binary subdivision approach are most apparent in linearized versions to be described below, we shall not discuss them further here.

4.2.3 Linearized Space Subdivision

While offering significant storage savings over exhaustive enumeration, octrees (and binary subdivision trees) are still quite large. This has led

researchers to investigate possibilities for compressing the octree representation further, and many alternative representations that replace the explicit tree structure with a pointer-free, linear data structure have been proposed. Obviously, such representations are also convenient for storing octrees into external storage.

Linear Octrees

Recall the numbering 0 ... 7 of octants of the octree. The octant numbers can be used to construct a *path address* for each node of the octree except the root. Clearly, the path address of a octree node of level i becomes a sequence of i digits $0, ..., 7$. A special "end-of-number" digit can be included to mark the tail of a number that has less digits than the maximal resolution.

The *linear octree* of Gargantini [42] is based on these observations. In her approach, the linear octree is simply the sorted list of path addresses to all black nodes. Hence, the linear octree that corresponds to the octree of Figure 4.2(c) is simply the list

$$\{03, 1X, 51, 53\}$$

where X denotes the "end-of-number" digit.

Another compact linear encoding of an octree is the so-called *DF-representation* [63]. It is formed by traversing the octree in preorder, and storing certain information of the nodes encountered. The encoding uses the alphabet "B", "W", and "(" denoting a black leaf, a white leaf, and an internal (nonleaf) node. Hence, the DF-representation of the octree of Figure 4.2(b,c) is the following string:

((WWWBWWWWBWWW(WBWBWWWWWW

As the alphabet has only three characters, two bits per each node are sufficient for the encoding.

Remarkably, mere linear representations are sufficient for many important algorithms. Boolean set operations on linear octrees end up in merging two linear strings of characters, a relatively straightforward operation. Algorithms for finding the nearest neighbor of a given node, for performing certain geometric operations, and for calculating connected components of objects also have been reported. For instance, reference [108] describes several algorithms for the two-dimensional case.

Bintree: Linear Binary Subdivision

Binary subdivision schemes lend themselves to very similar compact representations. In comparison with linear octrees, they are slightly smaller

because the number of leaves is smaller (compare Figures 4.2 and 4.3). Also, two leaves at the lowermost level of the tree under same parent can be encoded with one bit only because there are only two possibilities (black-white, white-black). These and other enhancements lead to linear representations having approximately one bit per tree node [115].

In three dimensions, Samet and Tamminen call the resulting solid representation a *bintree* [107,65]. Again, many algorithms can work without ever constructing an explicit tree. For instance, the paper [107] describes an algorithm for the evaluation of Boolean expressions of solid primitives. Display algorithms for bintrees are described by Koistinen *et al.* [65].

4.2.4 Geometric Search by Subdivision

Having an explicit octree available, it is very easy to check whether some particular location in the space of interest contains material or not. This point of view leads us to consider octrees and other space subdivision schemes as an efficient access method to geometric information.

Ordinarily, octree nodes only store a few bits of information encoding the material class of the cell. In the case of a geometric index, nodes may store (references to) other geometric entities, such as points, lines, or surfaces.

For the purposes of solid modeling, the prime example of this approach is the *extended octree* [7,22,86] that stores geometric elements of a boundary representation into an octree. As we interpret the extended octree as a hybrid representation scheme, we shall discuss its details in Chapter 7.

4.3 CELL DECOMPOSITIONS

Another approach to resolve the problems of exhaustive enumeration while preserving its nice properties is to use also other kinds of basic elements than just cubes. These schemes are called *Cell Decompositions.*

4.3.1 Representation of Cell Decompositions

A cell decomposition has a certain variety of basic *cell types* and a single combination operator *glue*. Individual cells are usually created as parameterized instances of cell types. Cells may be any objects that are topologically equivalent to a sphere (i.e., do not contain holes); in particular, they may include curved surfaces. A solid is modeled by means of a collection of semidisjoint cells, i.e., cells may "touch" each other along their bounding surfaces, but not have common interior points; see Figure 4.4 [35].

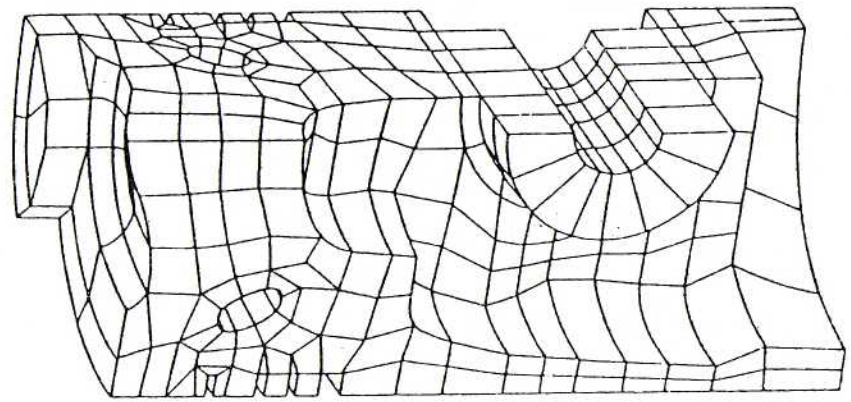

Figure 4.4 A cell decomposition.

A typical cell might be a curved polyhedron determined by 20 points. Eight points reside at its corners, and 12 points on the lines between the corners. Hence, each line has three points which define a quadratic curve, and each surface becomes a biquadratic surface patch determined by eight points (see Figure 4.5).

Typically, the collection of cells must satisfy also certain topological criteria in addition to being pairwise semidisjoint. Usually, any two cells are required either to be completely disjoint or meet in exactly one corner, along one line, or along one face. The case that two cells intersect otherwise is considered an incorrect cell decomposition. Such requirements make it quite difficult for a human being to construct valid cell decompositions directly.

In the important case of finite element models (FEM), the combination operation *glue* is usually represented implicitly through an encoding of "nodes" in the cell decomposition data structure. This is important for arranging the communication between a cell and its neighbors.

4.3.2 Properties of Cell Decompositions

Again, let us discuss the properties of cell decompositions according to the general scheme.

Expressive power: The modeling space is general and in the presence of cells with curved surfaces exact up to the degree of the cells, typically quadratic.

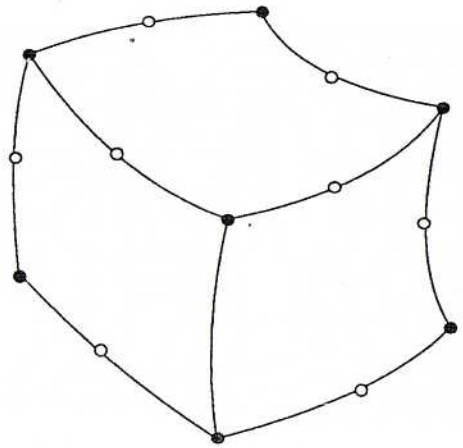

Figure 4.5 A quadratic cell.

Validity: The validity of a cell decomposition is hard to establish. While exhaustive enumeration and octree representation have structural properties that assure validity, a general cell decomposition is just an unordered set of cells. To check its validity, an intersection test for *every* pair of cells is necessary.

Unambiguity and uniqueness: A valid cell decomposition completely determines a solid. Cell decompositions are not unique.

Description languages: In general, it is very hard to create cell decompositions of interesting objects by direct mechanisms. Usually cell decompositions are created by conversion from other, more convenient representations.

Conciseness: Cell decompositions are relatively concise. The usual representation consists of a list of *nodes*, each being a named three-dimensional point, followed by a list of *elements*, each having a type code and a list of point numbers, whose interpretation is determined by the type code.

Closure of operations: Usually cell decompositions are used only as input to analysis algorithms that passively examine the model without modifying it. Nontrivial closed algorithms for cell decompositions (such as Boolean set operations) are either unknown or computationally unattractive.

Computational ease and applicability: Cell decompositions are indispensable as the input representation for computational algorithms. Finite element models (FEM) give a prime example of this.

The role of cell decompositions is clear from the above: they are auxiliary representations used for specific computations rather than as general independent solid models in their own right. We shall return to this later in Section 5.2.

PROBLEMS

4.1. Show that 7/8 of the nodes of an octree are leaves.

Hint: Use induction on the depth of the tree.

4.2. How would you classify an octree node against straight cylinder primitive?

4.3. Describe an algorithm for rotating an octree by a integer multiple of 90 degrees.

4.4. Describe algorithms for calculating (a) the set union and (b) the set difference of two octrees.

4.5. Describe an algorithm that can calculate Boolean set operations of two linear quadtrees represented by means of the DF-representation.

4.6. Design a data structure that can be used to represent a cell decomposition consisting of quadratic cells such as the one shown in Figure 4.5.

BIBLIOGRAPHIC NOTES

Octrees seem to have been invented independently by several workers. The basic works are by Hunter [59], Jackings and Tanimoto [60], Meagher [83,82], and Reddy and Rubin [93].

The major source of information on quadtrees and related data structures is the comprehensive survey [106] of Samet that also includes a large bibliography.

The survey of Shirari [112] offers lots of information on various representations of three-dimensional digital images primarirly from the point of view of image processing applications.

While accurate octree models of curved solids would be very large, it seems possible to use octrees for cases where somewhat rough models are appropriate. In addition to graphics, Yamaguchi *et al.* [135] describe the use of octree models for rough cutting of mechanical parts.

Cell decompositions are the primary data representation for the Finite Element Method. Standard references to FEM are [12,54].

FEM systems typically include a preprocessor that aids the generation of proper FEM meshes. The more powerful preprocessors are actually solid modelers of their own right that can be useful even outside the FEM context proper; for instance, see [23].

Figure 4.1, page 60 is by A. H. J. Christessen and has been previously published as the inside cover figure of *Computer Graphics*, Volume 14, Number 3. ©1980, Association for Computing Machinery, Inc. Reprinted by permission.

Figure 4.4, page 73 has been previously published as Figure 3 of the article "Interactive graphical CAD in mechanical engineering design" by W. S. Elliott in *Computer-Aided Design*, Volume 10, Number 2. ©1978, Butterworth & Co (Publishers) Ltd. Reprinted by permission.